

I researched Blockchain deployment, and the most critical thing is to be able to hashing algorithms. I have implemented this in GPU and validate it runs on Nvidia T4 GPU.

This CUDA-C program implements a parallelized version of the SHA-256 hashing algorithm, designed to compute hashes for multiple 64-byte input chunks concurrently using GPU threads. Each thread processes one chunk of input data, generating a unique 256-bit (32-byte) hash. This implementation demonstrates how the GPU's massive parallelism can accelerate hashing operations, commonly used in cryptographic applications.

Functionality:

1. SHA-256 Components:

- Includes helper functions for bitwise operations (e.g., rotations, logical operations) and message preparation
- Implements the SHA-256 transformation logic to compute hashes based on input data and constants.

2. Parallelized Kernel:

- The `sha256_kernel` function assigns each thread to process a 64-byte chunk of data, calculating its hash.
- Threads update global memory with the resulting 256-bit hashes.

3. Memory Management:

- Input data and output hashes are transferred between host (CPU) and device (GPU) memory for processing.
- Dummy data is used as input for demonstration purposes.

4. Output:

- Displays the computed hashes for the first few input chunks.

Use Cases:

- Cryptographic applications require SHA-256 hashing (e.g., blockchain, data integrity verification).
- Accelerated hashing for large-scale datasets in distributed systems.
- Benchmarks for GPU performance in cryptographic workloads.

This program is ideal for scenarios where high-throughput hashing is needed, leveraging the Computational power of GPUs.

This project will help existing and future data bytes students to leverage the hashing function library and use the same functionality consistently. In future if Dat bytes students decided to work on secure generative AI based solutions, they can leverage the same hashing library to moderate the prompt (request) and output of the LLM (response) to stop the LLM red IP leakage and/or eliminate language profanity due to the model drift and hallucinations. The only caveat is that the library must be used with Nvidia SW ecosystem.